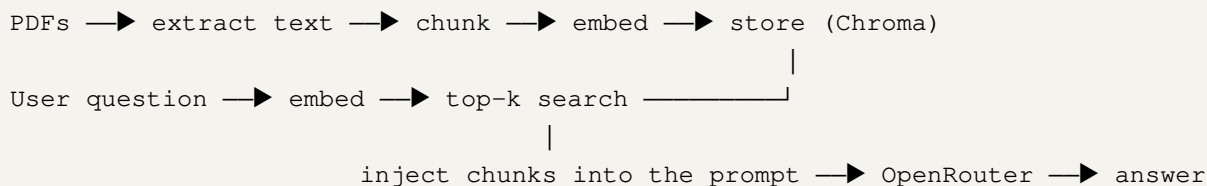


ArchMind RAG — Quick & Dirty Guide

Add Retrieval-Augmented Generation (RAG) to ArchMind so it can answer from **your own PDFs** (building codes, standards, lecture notes, manufacturer specs) instead of only the model's training data.

This is the *pragmatic* version — no fancy infra. Everything runs locally, free (embeddings are local, no extra API cost), and plugs straight into your existing `main.py`.



0. The two phases

RAG has two halves. Keep them separate in your head:

1. **Ingest (offline, run once per document set):** read PDFs → split into chunks → turn each chunk into a vector → save to a vector store on disk.
2. **Query (online, every chat turn):** turn the user's question into a vector → find the most similar chunks → paste them into the prompt → ask the model.

1. Install dependencies

```
# from the repo root (where main.py / pyproject.toml live)
uv add pypdf chromadb sentence-transformers
```

- **pypdf** — pull text out of PDFs.
- **sentence-transformers** — local embedding model (`all-MiniLM-L6-v2`, ~80MB, runs on CPU, no API key, free). First run downloads the model.
- **chromadb** — a tiny local vector database that persists to a folder. No server to run.

Why local embeddings? OpenRouter is great for chat but doesn't expose a clean embeddings endpoint. Local embeddings keep this free and offline. If you'd rather use OpenAI embeddings, see the note at the bottom.

2. Drop your PDFs somewhere

```
mkdir -p knowledge
```

```
# put your .pdf files in ./knowledge/  
# e.g. knowledge/ghana-building-code.pdf, knowledge/passive-design.pdf
```

3. The ingest script — `ingest.py`

Create `ingest.py` in the repo root. Run it whenever you add/change PDFs.

```
import os  
import glob  
from pypdf import PdfReader  
import chromadb  
from sentence_transformers import SentenceTransformer  
  
KNOWLEDGE_DIR = "knowledge"  
DB_DIR = "rag_store"          # Chroma persists here  
COLLECTION = "archmind"  
CHUNK_SIZE = 900              # characters, not tokens — dirty but fine  
CHUNK_OVERLAP = 150          # keeps sentences from being cut mid-thought  
  
embedder = SentenceTransformer("all-MiniLM-L6-v2")  
  
def pdf_to_text(path: str) -> str:  
    reader = PdfReader(path)  
    pages = []  
    for page in reader.pages:  
        pages.append(page.extract_text() or "")  
    return "\n".join(pages)  
  
def chunk_text(text: str) -> list[str]:  
    text = " ".join(text.split())          # collapse whitespace  
    chunks, start = [], 0  
    while start < len(text):  
        end = start + CHUNK_SIZE  
        chunks.append(text[start:end])  
        start = end - CHUNK_OVERLAP        # slide back for overlap  
    return [c for c in chunks if c.strip()]  
  
def main():  
    client = chromadb.PersistentClient(path=DB_DIR)  
    # wipe & rebuild so re-runs are idempotent  
    try:  
        client.delete_collection(COLLECTION)  
    except Exception:  
        pass  
    collection = client.create_collection(COLLECTION)  
  
    pdfs = glob.glob(os.path.join(KNOWLEDGE_DIR, "*.pdf"))
```

```

if not pdfs:
    print(f"No PDFs found in ./{KNOWLEDGE_DIR}/")
    return

docs, metas, ids = [], [], []
for pdf in pdfs:
    name = os.path.basename(pdf)
    text = pdf_to_text(pdf)
    chunks = chunk_text(text)
    print(f"{name}: {len(chunks)} chunks")
    for i, chunk in enumerate(chunks):
        docs.append(chunk)
        metas.append({"source": name, "chunk": i})
        ids.append(f"{name}-{i}")

print(f"Embedding {len(docs)} chunks...")
embeddings = embedder.encode(docs, show_progress_bar=True).tolist()

collection.add(documents=docs, embeddings=embeddings, metadatas=metas, ids=ids)
print(f"Done. Stored {len(docs)} chunks in ./{DB_DIR}/")

if __name__ == "__main__":
    main()

```

Run it:

```
uv run ingest.py
```

You now have a `rag_store/` folder holding your vectorized knowledge. **Add `rag_store/` and `knowledge/` to `.gitignore`** if the PDFs are large or private.

4. The retriever — `rag.py`

Create `rag.py`. This is what the chatbot calls at query time.

```

import chromadb
from sentence_transformers import SentenceTransformer

DB_DIR = "rag_store"
COLLECTION = "archmind"
TOP_K = 4                                # how many chunks to feed the model

_embedder = None
_collection = None

def _lazy():
    """Load the model + DB once, on first use."""

```

```

global _embedder, _collection
if _embedder is None:
    _embedder = SentenceTransformer("all-MiniLM-L6-v2")
    client = chromadb.PersistentClient(path=DB_DIR)
    _collection = client.get_collection(COLLECTION)
return _embedder, _collection

def retrieve(query: str, k: int = TOP_K) -> list[dict]:
    """Return the k most relevant chunks for a query."""
    embedder, collection = _lazy()
    q_emb = embedder.encode([query]).tolist()
    res = collection.query(query_embeddings=q_emb, n_results=k)
    hits = []
    for doc, meta, dist in zip(
        res["documents"][0], res["metadatas"][0], res["distances"][0]
    ):
        hits.append({"text": doc, "source": meta["source"], "distance": dist})
    return hits

def build_context(query: str, k: int = TOP_K) -> str:
    """Format retrieved chunks into a context block for the prompt."""
    hits = retrieve(query, k)
    if not hits:
        return ""
    blocks = []
    for h in hits:
        blocks.append(f"[Source: {h['source']}] \n {h['text']}")
    return "\n\n---\n\n".join(blocks)

```

5. Wire it into `main.py`

You already have `get_client()` and `iter_chunks()`. The only change is: **before calling the model, retrieve context and inject it**. Add one helper and use it in both the terminal loop and the `/chat` endpoint.

5a. Add a context-injection helper (top of `main.py`, after `SYSTEM_PROMPT`)

```

def inject_rag(messages: list) -> list:
    """Augment the latest user message with retrieved knowledge."""
    from rag import build_context  # local import so RAG is optional

    last_user = next(
        (m for m in reversed(messages) if m["role"] == "user"), None
    )
    if not last_user:
        return messages

    context = build_context(last_user["content"])

```

```

if not context:
    return messages

rag_note = (
    "Use the following retrieved reference material to answer the user's "
    "question when relevant. Cite the [Source: ...] name when you rely on it. "
    "If the material doesn't cover the question, say so and answer from your "
    "general knowledge – never invent a citation.\n\n"
    f"=== RETRIEVED MATERIAL ===\n{context}\n=== END MATERIAL ==="
)
# insert as a system message right before the last user turn
augmented = messages[:-1] + [
    {"role": "system", "content": rag_note},
    messages[-1],
]
return augmented

```

5b. Use it in the terminal loop

In `main()`, change the call from:

```
reply = stream_response(client, messages)
```

to:

```

reply = stream_response(client, inject_rag(messages))
messages.append({"role": "assistant", "content": reply})

```

Note: feed `inject_rag(messages)` to the model, but keep appending the *plain* user/assistant turns to `messages` so the retrieved block doesn't pollute history.

5c. Use it in the FastAPI `/chat` endpoint

Inside `serve()`, in the `chat()` function, change:

```

return StreamingResponse(
    iter_chunks(client, messages),
    media_type="text/plain; charset=utf-8",
)

```

to:

```

return StreamingResponse(
    iter_chunks(client, inject_rag(messages)),
    media_type="text/plain; charset=utf-8",
)

```

That's it. The website and terminal now both answer from your PDFs.

6. Test it

```
# 1. ingest
uv run ingest.py

# 2. terminal
uv run main.py
#   ask something only your PDF would know, e.g.
#   "What does the Ghana Building Code say about stair width?"

# 3. or the full stack
uv run main.py --serve      # terminal 1
cd frontend && npm run dev   # terminal 2
```

If answers cite [Source: your-file.pdf], RAG is working.

7. Knobs to tune (when "dirty" isn't good enough)

Problem	Fix
Answers miss relevant info	Raise <code>TOP_K</code> (4 → 6/8) or shrink <code>CHUNK_SIZE</code> .
Chunks cut mid-sentence	Raise <code>CHUNK_OVERLAP</code> , or switch to sentence-aware splitting (LangChain's <code>RecursiveCharacterTextSplitter</code>).
Retrieval is slightly off	Better embedding model: <code>BAAI/bge-small-en-v1.5</code> or <code>all-mpnet-base-v2</code> .
Irrelevant chunks sneak in	Filter by <code>distance</code> (drop hits above a threshold, e.g. > 1.0 for cosine).
Garbled text from scanned PDFs	<code>pypdf</code> can't read scans. Use OCR (<code>ocrmypdf</code> or <code>pytesseract</code>) first.
Slow first response	The embedder loads on first query — call <code>rag._lazy()</code> at server startup to warm it.
Want page numbers in citations	Track page index during <code>pdf_to_text</code> and store it in <code>metas</code> .

8. Alternatives (if you outgrow this)

- **Vector store:** swap Chroma for `pgvector` (Postgres), Pinecone, Qdrant, or Weaviate when you need scale/multi-user.
- **Embeddings via API** (instead of local): use OpenAI's `text-embedding-3-small`. Replace `embedder.encode(...)` with a call to `client.embeddings.create(model="text-embedding-3-small", input=chunks)` — note this needs an OpenAI key (OpenRouter doesn't proxy

embeddings cleanly), and it costs a tiny amount per token.

- **Framework:** LangChain or LlamaIndex wrap all of the above. Skip them until the hand-rolled version genuinely hurts — they add a lot of abstraction.
- **Re-ranking:** add a cross-encoder re-ranker (e.g. `bge-reranker-base`) between retrieval and the prompt for a quality bump on large corpora.

9. Gotchas / safety

- **Don't commit private PDFs or the vector store.** Add to `.gitignore`:

```
knowledge/  
rag_store/
```

- **RAG doesn't override ArchMind's safety boundaries.** The system prompt still forbids stamped structural/code-compliance answers — retrieved code text informs the explanation, it doesn't authorize a sign-off. Keep that framing.
- **Re-run `ingest.py`** every time PDFs change — the store is a snapshot.
- **Citations can still be wrong** if a chunk is misleading; the prompt tells the model not to fabricate `[Source]` tags, but spot-check important answers.